

Permission modes in Claude Code

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 5 of 12 · English notes

Session 5 (12 August 2026) resumed the interrupted frontend generation and opened with a recap of **permission modes**: how much Claude Code may do without stopping to ask you. The trainer listed four behaviours you should be able to contrast in writing: ask for every activity, accept edits automatically, plan-then-execute, and auto / “no stop” execution. Exact UI strings change; the control surface does not.

TRUST CURRENT CLAUDE CODE DOCS FOR LABELS

The recording says “ask,” “edit automatically,” “plan,” and “auto decide / no stop.” Current product docs may use names such as default, accept edits, plan, and bypass permissions. For the exam, describe *what is auto-approved* and *when you can still edit the to-do list*. Start at [Claude Code overview](#).

Why modes exist

Claude Code is an agent that can edit files, run bash, call MCP tools, and delete paths. Each of those is a *tool use* that in default mode wants consent. Modes are a policy for that consent: how often the loop pauses. They are not a different model and they are not a substitute for CLAUDE.md. They also are not “unsafe vs safe models” — Opus in auto mode is still auto mode.

The four behaviours from class

Classroom name	What Claude may do without asking	When to use it	Main risk
Ask (all activities)	Little: each edit, shell command, and MCP call can prompt.	Learning the tool; unfamiliar repos; production secrets nearby.	Slow; you become the bottleneck.
Edit automatically (accept edits)	File writes/edits; still likely to ask for risky shell or network.	You trust the plan and want fewer Save/Apply clicks.	Silent bad edits if you are not watching git diff.
Plan	Produce a to-do / plan first; you review and edit the plan; execution starts only after you accept.	Multi-step generation, Docker vs Playwright choices, anything you might want to rewrite.	Once execution has started, the trainer said you cannot edit that plan in-flight.
Auto / no stop (bypass-style)	Permission prompts handled automatically; the loop does not wait.	Long unattended generation after hooks and allow/deny lists exist.	Destructive bash (rm, git restore, etc.) unless pre-tool hooks block them (Module 06).

Plan mode is the one that intersects agents

A learner asked whether you can still intervene if agents run without interruption. The trainer split two ideas: the **orchestrator plan** (the to-do list: scaffold Vite, shared types matching OpenAPI, pages, tests...) is editable *while you are in plan mode, before execute*. If Playwright is on the list and you want Selenium, change the plan, then let agents run. After execute starts, you watch, skip, or stop — you do not quietly

rewrite the frozen plan. Human-in-the-loop is therefore front-loaded into planning, not sprinkled through every sub-agent turn.

- Plan mode is also an exam phrase in later scenario lists (“plan mode and direct execution”). Direct execution is the opposite: skip the reviewable to-do board and just go.
- Sub-agents can carry their own permission mode (e.g. bypass so a code-reviewer is not blocked on every grep). That is configured on the agent markdown, not only in the global picker (see M05-C03).
- Auto mode is exactly when Module 06 hooks (block dangerous commands, protect `.env`) become mandatory rather than nice-to-have.

Consent still belongs to you

Even in ask mode, session 4’s disclaimer stands: executed commands run with the end user’s consent. Switching to auto does not move legal or enterprise responsibility onto Anthropic. UST classroom rules still forbid pasting client secrets; auto mode will happily write whatever the model proposes into the working tree if you have not blocked those paths.

AUTO WITHOUT HOOKS

The 13 August session showed `rm` of a test file while auto was on; a pre-tool hook was supposed to deny it. Until that hook is proven in the *same* surface (terminal vs IDE chat), do not leave auto on a machine that can reach production credentials. Reload VS Code / webviews after changing `settings.json`.

How this showed up while finishing the app

Continuing code generation from the previous chat worked because the session still held the to-do list (context + plan), not because a single giant prompt was replayed. Permission mode decides whether each remaining to-do pauses. If generation is taking “a very long time,” that is usually agentic looping and verification, not a stuck permission dialog — unless you are in ask mode and missed a prompt in another panel.

Study check

Be ready to map a scenario: “developer wants to review architecture steps but not every file write” → plan then accept-edits, not full auto. “CI-like unattended refactor on a throwaway branch with protect-files hooks” → auto. “First day on Claude Code” → ask. Write the behaviour, then mention that the live toggle name should be checked in docs.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Permission modes, plan mode, and how the coding agent asks for tools.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Every permission prompt is a tool call under the hood.

- **Safety / usage policy**

<https://www.anthropic.com/legal/usage-policy>

Enterprise use still sits under Anthropic policy regardless of auto mode.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Hooks and allow/deny lists are the practical guardrail when bypass is on.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.